Campus Connect

Final Project Report

Cover Page

Project Title: Campus Connect

Course / Subject: Java Programming / Object-Oriented Programming Project

Student Name: Vityarthi

Academic Year: 2026

Institution: College / University

Submission Type: GitHub Repository + Project Report

1. Introduction

Campus Connect is an academic project developed in Java to simplify the management of student records and campus activities. The system is designed to reduce manual effort in handling student registration, event creation, seat booking, and generating simple reports. It provides a practical implementation of important Java concepts such as classes, objects, collections, validation, modular design, and basic application flow.

The aim of the project is to build a reliable and easy-to-use system that supports everyday administrative tasks in a college environment. The project is designed to be modular and expandable, allowing future integration with databases, login systems, and GUI interfaces.

2. Problem Statement

Colleges and educational institutions often manage student data and event participation manually. This usually leads to:

- duplicate or inconsistent records

- difficulty in tracking event capacity and booking status

- poor information visibility for administrators

- time-consuming manual record handling

A centralized digital system is needed to register students, manage events, reserve seats, and generate useful summaries. Campus Connect addresses this challenge by providing a simple and efficient platform for event and student management.

3. Objectives

The main objectives of the project are:

- To manage student records efficiently
- To create and organize university or campus events
- To allow students to book available seats for events
- To prevent duplicate bookings and invalid entries
- To provide summary reports for administration
- To implement a clean and scalable Java-based solution

4. Functional Requirements

The system includes the following major functional requirements:

4.1 Student Management

- Register new students
- Store student details such as name, department, email, and academic year
- View all registered students
- Search for students by ID

4.2 Event Management

- Create campus events
- Store event title, date, venue, and capacity
- Display all available events
- Check seat availability for each event

4.3 Booking Management

- Book a seat for a student in a selected event
- Prevent duplicate student-event bookings
- Cancel an existing booking
- Maintain booking history

4.4 Reporting

- Show total number of students
- Show total number of events
- Show total bookings and remaining seats
- Generate simple summaries for administration

5. Non-Functional Requirements

The project also specifies the following non-functional requirements:

- Performance: operations should respond quickly for small and medium-sized data sets.
- Security: invalid input, duplicate records, and invalid operations should be rejected.
- Usability: the application uses a clear console-based menu for easy interaction.
- Reliability: the system handles invalid data and business rule violations with clear messages.
- Maintainability: separate classes and service layers make the code easier to extend.
- Error Handling: exceptions are used for invalid entries and invalid operations.

6. System Architecture

The application follows a layered architecture:

1. Model Layer
- Student
- Event
- Booking

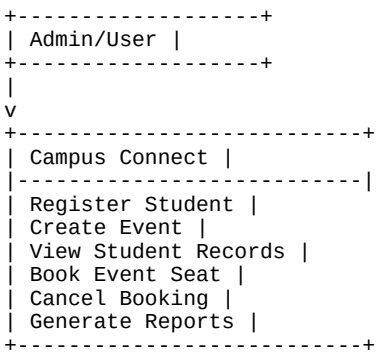
2. Service Layer
- StudentService
- EventService
- ReportService

3. User Interface Layer
- CampusConsole

This layered structure keeps responsibilities separate and improves maintainability. Business logic is not mixed with user interaction, making the project easier to expand in future versions.

7. Design Diagrams

7.1 Use Case Diagram



7.2 Workflow Diagram

```
Start
|
v
Display main menu
|
+--> Add Student
+--> Add Event
+--> List Students
+--> List Events
+--> Book Seat
+--> Cancel Booking
+--> Generate Reports
+--> Exit
```

7.3 Sequence Diagram

```
Admin -> CampusConsole : Select Book Seat
CampusConsole -> EventService : bookSeat(studentId, eventId)
EventService -> StudentService : validate student exists
StudentService --> EventService : valid student
EventService -> Event : decrement availableSeats
EventService -> Booking : create booking record
EventService --> CampusConsole : booking confirmation
CampusConsole --> Admin : display result
```

7.4 Class Diagram

```
+----------------+ +----------------+ +----------------+
| Student | | Event | | Booking |
+----------------+ +----------------+ +----------------+
| - studentId | | - eventId | | - bookingId |
| - name | | - title | | - studentId |
| - department | | - date | | - eventId |
| - email | | - venue | | - bookedAt |
| - year | | - capacity | | - status |
+----------------+ | - available | +----------------+
| +----------------+
| |
+------------------------------+
|
+----------------------+
| StudentService |
+----------------------+
| registerStudent() |
| getAllStudents() |
+----------------------+

+----------------------+
| EventService |
+----------------------+
| createEvent() |
| bookSeat() |
| cancelBooking() |
+----------------------+
```

7.5 ER Diagram

```
Student

studentId (PK)
name
department
email
year

Event

eventId (PK)
title
date
venue
capacity
availableSeats

Booking
```

```
bookingId (PK)
studentId (FK)
eventId (FK)
bookedAt
status
```

8. Design Decisions and Rationale

The following design decisions were made to align with good software engineering practices:

- Modular class design: Each entity and service handles a focused responsibility.
- In-memory data management: Java collections are used to store students, events, and bookings.
- Validation-first approach: invalid input is rejected before processing business logic.
- Console-based interface: keeps the project simple, readable, and compatible with academic evaluation.
- Separation of concerns: logic is split between model, service, and UI layers.

These decisions ensure the project is easy to understand, maintain, and extend into a database or graphical application later.

9. Implementation Details

The implementation was carried out using Java 17 and Maven. The project structure consists of model classes, service classes, and the console interface.

Model Components

- Student: stores the student identity and profile data
- Event: stores event details and seat capacity
- Booking: stores booking status and time

Service Components

- StudentService: handles student validation and registration
- EventService: handles event creation, booking, and cancellation
- ReportService: generates summary reports

User Interface

- CampusConsole: menu-driven console application used for all system operations

Sample Functionality

The console application supports:

- registering students

- creating events
- listing records
- booking and cancelling seats
- generating reports

10. Screenshots / Results

The application output is a console-based interface with a menu system. A sample interaction looks like this:

```
Welcome to Campus Connect

=== Campus Connect Menu ===
1. Register student
2. Create event
3. List students
4. List events
5. Book seat
6. Cancel booking
7. View my bookings
8. Generate reports
0. Exit
Choose an option:
```

This confirms that the system provides the required input/output structure and a logical workflow for user interaction.

11. Testing Approach

The project includes automated JUnit tests to validate key behavior. The tests cover:

- student registration validation
- event booking seat reduction

Testing Command

```
mvn test
```

Test Result

The project successfully builds and the unit tests pass without errors.

12. Challenges Faced

Some of the main challenges encountered during the project were:

- designing a clean and modular structure from scratch
- maintaining consistent validation rules across services
- ensuring that seat booking and cancellation logic remained correct
- balancing simplicity with academic requirements for documentation and design

These challenges were resolved by structuring the code into layered components and validating the logic through tests.

13. Learnings and Key Takeaways

This project strengthened understanding of:

- Java object-oriented programming
- modular software design
- data handling with Java collections
- validation and exception handling
- service-based architecture
- project documentation and report writing

It also demonstrated how a small software application can be built from requirements into a working, testable system.

14. Future Enhancements

The project can be extended in the future with:

- MySQL database integration
- login and role-based access control
- GUI support using Java Swing or JavaFX
- cloud-based deployment
- QR code check-in for events
- dashboard analytics and charts

15. References

- Java Documentation: https://docs.oracle.com/javase/
- Maven Documentation: https://maven.apache.org/
- JUnit 5 Documentation: https://junit.org/junit5/
- Oracle Java Tutorials: https://docs.oracle.com/javase/tutorial/

16. Conclusion

Campus Connect successfully demonstrates how a simple Java application can solve a real-world college management problem. The combination of object-oriented design, modular services, validation logic, and reporting features makes it a strong academic project aligned with the project requirements. It is simple to understand, easy to run, and suitable for further expansion into a database-driven or GUI-based system.